MIVA OPEN UNIVERSITY

Project Report & Defence Documentation

Course: COS 106 — Introduction to Web Technologies

Multi-Page Personal Portfolio & Interactive Academic Web Application

Jehoiada Enyinyigocho Abu
Bachelor of Software Engineering
Student Repository: github.com/jehoabu/cos_106_portfolio_project

Academic Year 2026

Abstract

This report provides a formal technical description and architectural defence for a multi-page web portfolio and interactive academic planner application developed for evaluation in **COS 106: Introduction to Web Technologies** at Miva Open University. The system is constructed using semantic HTML5, modern CSS3 (Flexbox and CSS Grid), and modern vanilla JavaScript (ES6+). The document highlights design rationales, security mitigations against Cross-Site Scripting (XSS), event-driven DOM manipulation strategies, form validation logic, responsive design patterns, and codebase explanations designed to support oral project defence.

Contents

1 Project Overview **3**
1.1 Introduction . . . 3
1.2 Purpose and Objectives . . . 3
1.3 Major Features and Functionalities . . . 3
1.4 Technology Stack Matrix . . . 3

2 Screenshots and Page Layout Explanations **4**
2.1 Homepage Interface (`index.html`) . . . 4
2.2 About Page & Academic Table (`about.html`) . . . 4
2.3 Projects Showcase & Grid Layout (`projects.html`) . . . 5
2.4 Academic Planner Web Application (`planner.html`) . . . 5
2.5 Contact Form & Alert Banner (`contact.html`) . . . 6

3 JavaScript Concepts & Implementation Analysis **7**
3.1 Event-Driven Architecture & Default Prevention . . . 7
3.2 XSS Prevention via Safe DOM Node Creation . . . 7
3.3 Performance Optimization: Event Delegation . . . 7
3.4 Client-Side Validation via Regular Expressions (RegEx) . . . 7
3.5 Separation of Concerns: State-Based Styling . . . 7

4 Codebase Breakdown & Defence Preparation **8**
4.1 Academic Planner Logic (`js/planner.js`) 8
4.2 Contact Form Validation Logic (`js/contact.js`) 9
4.3 Modular CSS Layout & Responsive Grid (`css/style.css` & `css/forms.css`) 10
4.4 Anticipated Defence Questions & Ideal Answers 11

5 Project Links & Deployment Information **12**
5.1 Public Web URLs 12
5.2 Deployment Pipeline 12
5.3 Verification Summary 12

1 Project Overview

1.1 Introduction

The web application is a responsive, accessible, multi-page personal portfolio integrated with interactive client-side web tools. Designed and developed from the ground up, the website serves as both a digital professional presence and a functional showcase of software engineering competencies acquired during the COS 106 module.

1.2 Purpose and Objectives

The primary objectives of this project are:

1. **Demonstrate Competency in Web Core Technologies:** Implement modern standards across structural markup (HTML5), visual aesthetics and responsive layouts (CSS3), and programmatic client-side behavior (JavaScript ES6+).
2. **Provide Interactive Student Utility:** Deliver a real-world tool—an Academic Task Planner—that allows students to manage coursework deadlines dynamically.
3. **Ensure Accessibility and Security:** Adhere to WCAG accessibility guidelines (semantic HTML, proper ARIA attributes, explicit focus states) and safe coding practices (XSS defense via DOM text nodes).

1.3 Major Features and Functionalities

The system comprises five core interlinked web pages:

- **Homepage (`index.html`):** Features a hero section, personal photography styled with CSS circular masking, welcome message, and biographical introduction.
- **About Page (`about.html`):** Contains an academic qualification table built using semantic `<thead>` and `<tbody>` tags, paired with a Flexbox skills badge cloud.
- **Projects Gallery (`projects.html`):** Formatted using responsive CSS Grid (`repeat(auto-fit, minmax(250px, 1fr))`), containing multimedia video elements and project cards with interactive hover effects.
- **Academic Planner (`planner.html`):** An interactive task management application supporting real-time task creation, completion toggling, and task deletion using Event Delegation.
- **Contact Portal (`contact.html`):** Features custom RegEx validation for email formatting and digit-only phone inputs, utilizing dynamic ARIA alert banners for accessible client feedback.

1.4 Technology Stack Matrix

Table 1 outlines the technological decisions powering the application.

Table 1: Technology Stack Summary

Layer	Technology / Module	Core Purpose
Markup	Semantic HTML5	Document structure, accessibility, forms, tables, media.
Styling	CSS3 (Modular Architecture)	Visual layout, responsive grid, flexbox, transitions.
Typography	Google Montserrat Font	Modern, highly readable sans-serif design system.
Scripting	Vanilla JavaScript (ES6+)	Dynamic DOM updates, event handling, input validation
Hosting	GitHub Pages	Production deployment and public web access.
Version Control	Git & GitHub	Code revision management and project hosting.

2 Screenshots and Page Layout Explanations

This section details the user interface components across the application. (Note: Frame placeholders illustrate the layout positioning of actual browser screenshots required during defence presentation).

2.1 Homepage Interface (`index.html`)

[Screenshot Placeholder: Homepage (`index.html`)]
Capturing Header Navigation, Circular Profile Picture, Welcome Message, and Biography Card.

Figure 1: Homepage UI structure showcasing navigation and bio container.

- **Section Description:** The homepage serves as the primary landing hub. It features a top navigation header (`<nav id="nav_menu">`), a profile photo wrapped in `<div id="name_picture">`, an introductory text block, and a structured biography box.
- **Implementation Details:** The profile image is formatted using CSS `border-radius: 50%` and `object-fit: cover` to ensure an exact circular mask without aspect ratio distortion. The navigation items use Flexbox (`display: flex; gap: 25px;`) for spacing.

2.2 About Page & Academic Table (`about.html`)

- **Section Description:** Displays educational background and technical skills.
- **Implementation Details:** The academic history uses explicit HTML5 table structures (`<table>`, `<thead>`, `<tbody>`, `<th>`, `<td>`). Alternating rows feature subtle background shading using the CSS selector `tr:nth-child(even)`. The skills list (`<ul class="skills">`) leverages `flex-wrap: wrap` to transform list items into pill-style tags that adjust gracefully across mobile screens.

[Screenshot Placeholder: About Page (`about.html`)]
Displaying Education History Table and Technical Skills Tag Cloud.

Figure 2: About page showing semantic table styling and flex wrapped skill tags.

[Screenshot Placeholder: Projects Gallery (`projects.html`)]
Illustrating 2-column Grid Layout on Desktop, Card Hover Transformations, and Video Control Interface.

Figure 3: Projects page featuring CSS Grid card arrangement and video player.

2.3 Projects Showcase & Grid Layout (`projects.html`)

- **Section Description:** Displays software projects in an interactive card layout alongside embedded multimedia.
- **Implementation Details:** Container alignment is achieved using CSS Grid: `grid-template-columns: repeat(auto-fit, minmax(250px, 1fr))`. Cards lift on hover via `transform: translateY(-5px)` combined with a smooth `box-shadow` transition. Multimedia integration is fulfilled using an HTML5 `<video>` element with explicit controls, height, and width parameters.

2.4 Academic Planner Web Application (`planner.html`)

[Screenshot Placeholder: Academic Planner Application (`planner.html`)]
Showing Input Form, Date Selector, Active Tasks, Checkbox Toggling, and Delete Actions.

Figure 4: Interactive Academic Planner interface demonstrating dynamic task list items.

- **Section Description:** Provides a live interface for adding, completing, and removing academic tasks.
- **Implementation Details:** Styled using `forms.css`. Task list items (`<ul id="task-list">`) render as card rows using Flexbox (`justify-content: space-between`). Completed items

dynamically receive the `.completed` class, applying a strikethrough decoration.

2.5 Contact Form & Alert Banner (`contact.html`)



Figure 5: Contact page illustrating input form controls and alert status states.

- **Section Description:** Collects user inquiries while executing client-side validation checks.
- **Implementation Details:** Includes inputs for name, email (`type="email"`), phone (`type="tel"`), and message (`<textarea>`). Form inputs display visible focus indicators (`outline: 2px solid #1098AD`). Dynamic alerts are injected into an accessible feedback element featuring `role="alert"`.

3 JavaScript Concepts & Implementation Analysis

JavaScript drives the interactive components of the portfolio. This section breaks down the foundational concepts utilized across `planner.js` and `contact.js`.

3.1 Event-Driven Architecture & Default Prevention

All user interactions start by listening to DOM events. To prevent traditional HTML forms from triggering a full page reload upon submission, event listeners execute `event.preventDefault()`.

3.2 XSS Prevention via Safe DOM Node Creation

A critical security requirement in web development is preventing Cross-Site Scripting (XSS). In `planner.js`, user input is never injected directly via `innerHTML`. Instead, elements are created explicitly and assigned text via `.textContent`:

- **Mechanism:** `document.createElement('span')` creates an isolated node.
- **Safe Assignment:** Assigning string data to `textContent` instructs the browser engine to treat the value strictly as literal character data. If a user submits malicious markup (e.g., `<script>alert('xss')</script>`), the browser renders the characters harmlessly on screen rather than compiling them as executable code.

3.3 Performance Optimization: Event Delegation

Instead of attaching separate event listeners to every individual task item's "Delete" button or checkbox (which consumes memory and fails to work for newly added items), the application utilizes **Event Delegation**:

- A single event listener is attached to the static parent container (`<ul id="task-list">`).
- When a click occurs inside the list, the handler inspects `event.target`.
- If `event.target.tagName === 'BUTTON'`, the parent `<li>` element is removed via `event.target.parentE`
- If `event.target.type === 'checkbox'`, the parent `<li>` toggles the `.completed` class.

3.4 Client-Side Validation via Regular Expressions (RegEx)

In `contact.js`, input strings are sanitized with `.trim()` and evaluated against regular expression patterns:

1. **Email Validation Pattern:** `/^[^\s@]+@[^\s@]+\.[^\s@]+$/` ensures the string contains non-whitespace characters surrounding an `@` symbol and a valid domain extension.
2. **Phone Validation Pattern:** `/^\d+$/` restricts input exclusively to numerical digits ($0 - 9$).

3.5 Separation of Concerns: State-Based Styling

Rather than manipulating inline style attributes directly within JavaScript (e.g., `element.style.color = 'red'`), script routines modify element state via CSS class assignments:

- Validation errors trigger `messageDisplay.className = 'alert alert-error'`.
- Successful submissions trigger `messageDisplay.className = 'alert alert-success'`.

This maintains a strict separation: JavaScript manages business logic and state, while CSS governs visual presentation.

4 Codebase Breakdown & Defence Preparation

This section presents key excerpts from the project codebase, accompanied by technical explanations to assist during oral examination.

4.1 Academic Planner Logic (`js/planner.js`)

```
// Select DOM elements
const taskForm = document.getElementById('task');
const taskInput = document.getElementById('task_input');
const deadlineInput = document.getElementById('deadline');
const taskList = document.getElementById('task-list');

// Add Task Event Listener
taskForm.addEventListener('submit', function(event) {
    event.preventDefault(); // Prevent page refresh

    const taskText = taskInput.value.trim();
    const deadlineText = deadlineInput.value.trim();

    if (!taskText) return; // Prevent blank submissions

    // Safely build DOM elements
    const newListItem = document.createElement('li');

    const textSpan = document.createElement('span');
    textSpan.textContent = `${taskText} (Due: ${deadlineText}) `;

    const checkbox = document.createElement('input');
    checkbox.type = 'checkbox';

    const deleteBtn = document.createElement('button');
    deleteBtn.type = 'button';
    deleteBtn.textContent = 'Delete';

    // Assemble components
    newListItem.appendChild(textSpan);
    newListItem.appendChild(checkbox);
    newListItem.appendChild(deleteBtn);

    taskList.appendChild(newListItem);

    // Reset inputs
    taskInput.value = '';
    deadlineInput.value = '';
});

// Event Delegation for Task Actions
taskList.addEventListener('click', function(event) {
    if (event.target.tagName === 'BUTTON') {
        event.target.parentElement.remove();
    }
    if (event.target.type === 'checkbox') {
        const listItem = event.target.parentElement;
        listItem.classList.toggle('completed');
    }
});
```

Listing 1: Dynamic Task List Creation and Event Delegation

Key Defence Explanation Points for `planner.js`:

- **Why use `event.preventDefault()`?** HTML forms naturally attempt to send an HTTP GET/-POST request to a server, causing a page refresh. Calling this method halts default browser processing, enabling seamless single-page interactivity.
- **Why build elements manually instead of setting `innerHTML`?** Setting `innerHTML` directly parses string input as raw markup. Manually constructing elements and binding text using `.textContent` ensures user data cannot execute malicious script vectors.

4.2 Contact Form Validation Logic (`js/contact.js`)

```
const contactForm = document.getElementById('contact-form');

// Create dynamic alert container
const messageDisplay = document.createElement('div');
messageDisplay.setAttribute('role', 'alert');
messageDisplay.className = 'alert alert-hidden';
contactForm.prepend(messageDisplay);

contactForm.addEventListener('submit', function(event) {
    event.preventDefault();

    // Reset alert display state
    messageDisplay.className = 'alert alert-hidden';
    messageDisplay.textContent = '';

    const name = document.getElementById('name').value.trim();
    const email = document.getElementById('email').value.trim();
    const phone = document.getElementById('phonenumber').value.trim();
    const message = document.getElementById('message').value.trim();

    // Empty field validation
    if (!name || !email || !phone || !message) {
        showError("Please fill out all fields before submitting.");
        return;
    }

    // RegEx validation
    const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    if (!emailPattern.test(email)) {
        showError("Please enter a valid email address (e.g., name@example.com).");
        return;
    }

    const phonePattern = /^\d+$/;
    if (!phonePattern.test(phone)) {
        showError("Your phone number must contain only digits (no spaces or dashes).");
        return;
    }

    showSuccess(`Thank you, ${name}! Your message has been sent successfully.`);
    contactForm.reset();
});

function showError(errorText) {
    messageDisplay.textContent = errorText;
    messageDisplay.className = 'alert alert-error';
}

function showSuccess(successText) {
```

```
    messageDisplay.textContent = successText;
    messageDisplay.className = 'alert alert-success';
}
```

Listing 2: Client-Side Form Validation and Dynamic Alerts

Key Defence Explanation Points for `contact.js`:

- **Why add `role="alert"`?** This attribute creates an ARIA live region. When text content is inserted into this node, screen readers automatically interrupt to read the message aloud, granting equal accessibility to visually impaired users.
- **What does `.trim()` do?** It strips leading and trailing whitespace characters from input strings, preventing users from passing validation by simply submitting blank space keypresses.

4.3 Modular CSS Layout & Responsive Grid (`css/style.css` & `css/forms.css`)

```
/* Global Box-Sizing Reset */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

/* Responsive CSS Grid Gallery */
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 25px;
  margin-top: 20px;
}

/* Card Component & Transitions */
.card {
  background-color: #f4f7f6;
  border: 1px solid #ddd;
  border-radius: 8px;
  overflow: hidden;
  display: flex;
  flex-direction: column;
  transition: transform 0.3s ease, box-shadow 0.3s ease;
}

.card:hover {
  transform: translateY(-5px);
  box-shadow: 0 8px 15px rgba(0, 0, 0, 0.1);
}
```

Listing 3: Excerpts from Layout and Card Grid Stylesheet

Key Defence Explanation Points for CSS Architecture:

- **How does `repeat(auto-fit, minmax(250px, 1fr))` work?** This single rule builds fully responsive columns without needing hardcoded media queries. It dictates that columns must stretch equally (`1fr`), but cannot shrink below `250px`. If the screen width contracts below 250px, cards automatically wrap down to the next row.

- **Why use `box-sizing: border-box`?** By default, adding padding or borders expands an element's calculated width. Setting `border-box` forces the browser to calculate padding and borders *inside* the specified width, preventing layout breakage.

4.4 Anticipated Defence Questions & Ideal Answers

Defence Q&A 1: Layout Engine Selection

Question: Why did you use Flexbox for the navigation bar, but CSS Grid for the projects page?
Ideal Answer: Flexbox is optimized for one-dimensional layouts (aligning items along a single row or column), making it ideal for the horizontal navigation bar. CSS Grid is designed for two-dimensional layouts, making it the perfect tool for structuring the project gallery cards into responsive rows and columns simultaneously.

Defence Q&A 2: Dynamic Listener Scalability

Question: What is Event Delegation and why did you use it on the Academic Planner?
Ideal Answer: Event Delegation is an architecture pattern where a single event listener is attached to a parent element rather than attaching multiple listeners to child elements. We use it on the planner's `<ul id="task-list">` container so that whenever new tasks are added dynamically to the DOM, their delete buttons and checkboxes are automatically handled without having to bind new event listeners repeatedly.

Defence Q&A 3: Architectural Modularization

Question: Why did you separate your styling into `style.css` and `forms.css`?
Ideal Answer: To adhere to the principle of modularity and separation of concerns. `style.css` handles global structure, typography, navigation, grid layout, and themes used across all pages. `forms.css` contains dedicated styles specific to form inputs, alert banners, and planner list items. This prevents stylesheet pollution and makes code maintenance cleaner.

5 Project Links & Deployment Information

The completed project is hosted live on the web and maintained in a public GitHub source control repository.

5.1 Public Web URLs

- **Live Hosted Portfolio:**
  https://jehoabu.github.io/cos_106_portfolio_project/
- **Source Code GitHub Repository:**
  https://github.com/jehoabu/cos_106_portfolio_project

5.2 Deployment Pipeline

Deployment is executed using **GitHub Pages**:

1. Local code changes are staged, committed, and pushed to the `main` branch of the repository.
2. GitHub Pages monitors the `main` branch `/(root)` directory.
3. An automated deployment pipeline compiles the static HTML, CSS, JavaScript, and multimedia assets, publishing the site to a production content delivery network (CDN).

5.3 Verification Summary

All links, video components, dynamic scripts, responsive styles, and accessible controls have been validated across modern desktop and mobile browser engines (Chromium, WebKit, and Gecko).